UNIVERSITÉ DE ROUEN NORMANDIE

UFR Sciences et Techniques



Licence Informatique – L2
Année 2025 – 2026

PROJET ALGORITHMIQUE 3

Génération de textes aléatoires —RANDTEXT

Réalisé par :

GRAINE Rayane Aghiles

BENADJAOUD Accyl

Encadrée par :

R.GROULT

Date de rendu :

28 Mai 2026

SOMMAIRE

Table des matières

1. Introduction 3

2. Architecture Générale et Structure du Projet..... 3

2.1 Architecture du projet.....3

2.2 Description de l'implantation et structures de données.....3

2.3 Fonctionnement général du programme 4

3. Choix d'Implantation Spécifiques 5

3.1 Choix de traitement des options sur la ligne de commande 5

3.2 Tableau dynamique pour les successeurs 5

4. Points forts 5

5. Analyse Théorique de Complexité 6

6. Limites..... 6

7. Conclusion 7

8.Annexes 8

Annexe A : Liens entre les modules et schéma implantation 8

Annexe B : Tableau récapitulatif des options prises en charge.....9

Annexe C : Tableau récapitulatif des statistiques.....10

1. Introduction

Le projet ***randtext*** propose une implantation modulaire stricte et performante en temps et en espace de l'algorithme de génération de texte aléatoire, en utilisant des tables de hachage, des structures de données linéaires et des tableaux dynamiques.

2. Architecture Générale et Structure du Projet

2.1 Architecture du projet

Dans Annexe A, figure 1, un schéma représentant l'architecture du projet et les différents liens entre les modules.

Le projet s'organise autour du dossier racine *graineray_projet/*. Le sous-dossier *doc/* contient le rapport *randtext.pdf* ; le *makefile* à la racine est destiné à la confection de l'archive de rendu. Le sous-dossier *randtext/randtext_test/* contient la fonction principale *main.c*, chargée de la lecture du source, de la construction et de la génération du texte, ainsi qu'un *makefile* dédié à la production de l'exécutable final. Le sous-dossier *randtext/source/randtext/* constitue le cœur algorithmique : l'interface *randtext.h* définit les fonctions liées à la génération du texte, tandis que *randtext.c* en propose l'implantation et *randtext_ip.h* détaille la complexité de chaque fonction implantée. Au sein de ce répertoire, le sous-dossier *option/* regroupe la gestion des options: *define.h* centralise les macro-constantes et macros-fonctions, le sous-module *help.h/.c* réunit les fonctions d'affichage des messages informationnels, tandis que *option.h/.c* et *option_ip.h* prennent en charge l'analyse de la ligne de commande.

Enfin, *randtext/source/module_util/* contient les trois modules utilitaires fournis par l'enseignant : *holdall/*, *hashtable/* et *malloccp/*.

2.2 Description de l'implantation et structures de données

Les tables de hachage (*hashtable*)

- **h_word** (table des mots uniques) : Elle utilise la fonction de hachage *str_hashfun* et la comparaison standard *strcmp*, les champs *keyref* et *valref* qui représentent le couple clé-valeur de la table de hachage pointent sur la même zone mémoire.
- **h_keyword** (table des couples clé-valeurs) : Cette table associe un contexte spécifique (la clé) à la liste de ses mots successeurs. Elle utilise une fonction de hachage dédiée (*context_hash*) et une fonction de comparaison (*context_cmp*) capables d'évaluer des tableaux de pointeurs.

Les fonctions des tables de hachage on était omise volontairement sur le schéma faute de place.

Les fours-tous (Holdall)

- ***hold_word*** (lié à *h_word*) : stocke dans l'ordre d'apparition tous les mots uniques.
- ***hold_key*** (lié à *h_keyword*) : conserve l'ordre de création de toutes les clés de contexte.

Ces deux conteneurs permettent l'affichage formaté en temps linéaire et simplifient la procédure de nettoyage en fin de programme : il suffit d'appliquer une fonction de libération sur chaque élément du fourre-tout pour désallouer proprement les chaînes, les tableaux de contextes et les structures *ctrvalref* sans aucune fuite.

Les structures de données internes (module *randtext*)

Les éléments insérés dans la table *h_word* nécessitent des structures de données spécifiques, dissimulées derrière l'interface *randtext.h* :

- **Les clés de contexte (*keyarray*) :** un contexte est représenté par un tableau alloué dynamiquement dont la taille correspond à l'ordre n de génération. Ce tableau stocke les pointeurs des n derniers mots lus.
- **Les listes de successeurs (*ctrvalref*) :** implantées sous la forme d'une structure constituée de trois champs :
  - *cap* : capacité maximale allouée.
  - *length* : nombre d'éléments réellement insérés.
  - *valarray* : tableau dynamique de pointeurs avec possibilité de réallocation.

2.3 Fonctionnement général du programme

L'explication suivante s'appuie sur le schéma mémoire présenté en Annexe A (*figure 2*), qui illustre l'organisation des structures du programme ainsi que les relations entre les différentes zones mémoire utilisées durant l'exécution :

Le programme commence par analyser l'ensemble des arguments de la ligne de commande. Chaque argument est examiné afin de déterminer s'il correspond à une option ou à un fichier. Lorsqu'un argument est identifié comme une option, la configuration interne du programme est mise à jour avec la valeur correspondante. Lorsqu'un argument correspond à un fichier, le programme tente d'ouvrir ce dernier afin de lire son contenu. Si aucun fichier n'est fourni, le programme s'attend à lire le texte sur l'entrée standard. Une fois le fichier ouvert, le texte est lu mot par mot. La taille maximale d'un mot est fixée soit à partir de l'option choisie par l'utilisateur, soit à partir du paramètre par défaut du programme.

Chaque mot lu est ensuite recherché dans la table de hachage *h_word* (représentée en vert sur le schéma mémoire) au sens de la fonction de comparaison standard *strcmp*. Si le mot existe déjà, aucune nouvelle allocation n'est effectuée. Dans le cas contraire, une nouvelle chaîne de caractères est allouée dynamiquement afin de stocker ce mot. Le mot est alors inséré dans *h_word* ainsi que dans la structure *hold_word*.

Après l'enregistrement du mot, le programme construit une clé nommé *keyarray* (représenté en jaune sur le schéma). Initialement, cette clé correspond à un tableau contenant uniquement des mots vides (ε, ε, ..., ε), dont la taille est égale à la valeur *order*, une fois le premier mot lu, il est ajouté comme successeur de cette clé. Les clés sont enregistrées dans la table de hachage *h_keyword* (en rouge sur le schéma, par soucis de représentation, lfmax a été fixé à 3 sur le schéma alors que dans notre implantation, il est fixé à 2). Pour chaque clé, on alloue un contrôleur *ctrvalref* (représentée en mauve sur le schéma) qui va lui être associé dans *h_keyword*, ce dernier possède un champ *valarray* qui représente un tableau dynamique (représenté par un tableau de couleur bleu foncé, possédant un coté pointillé faisant référence à la notion de réallocation dynamique) de pointeurs vers les mots successeurs (représenter par les flèches de couleur bleu cyan). Pour les successeurs, aucune nouvelle allocation de chaîne de caractères n'est réalisée. Le programme réutilise directement les mots déjà présents en mémoire afin d'éviter toute duplication des chaînes de caractères.

La clé créée est également enregistrée dans la structure *hold_key*. Ainsi, toutes les clés présentes dans *h_keyword* possèdent également une référence conservée dans *hold_key*.
Si la clé était déjà présente dans h_keyword, le mot lu s'ajoute à la liste de ses successeurs.

Le programme génère ensuite la clé suivante par décalage à gauche des éléments de la clé précédente et en insérant le mot lu à la dernière position du tableau. Aucun mot n'est réalloué

durant cette opération : seules les adresses mémoire des mots déjà existants sont manipulées. Les flèches de couleur *magenta* du schéma illustrent ce partage des pointeurs entre les différentes structures. Une nouvelle clé est alors créée sous la forme d'un tableau de pointeurs vers les mots correspondants. Ce processus est répété jusqu'à la fin du texte source. La dernière clé générée reçoit ε comme successeur supplémentaire, marquant la fin du texte.

Une fois la lecture terminée, la phase de génération du texte peut commencer. Durant cette phase, un unique tableau de taille order est alloué puis initialisé avec des mots vides, correspondant à la clé initiale (ε, ε, ..., ε). Le programme sélectionne alors aléatoirement l'un des successeurs associés à cette clé. Ce mot est affiché sur la sortie standard et permet de construire une nouvelle clé grâce à un décalage à gauche du tableau et l'insertion de ce mot en dernière position. Le même mécanisme est répété jusqu'à ce que le successeur tiré soit le mot vide ε, indiquant la fin de la génération, ou que la limite du nombre de mots produits soit atteinte.

3. Choix d'Implantation Spécifiques

3.1 Choix de traitement des options sur la ligne de commande

Les options *--help, --man, --options, --usage* et *--version* provoquent l'affichage correspondant depuis *getoption* et non depuis les fonctions elles-mêmes, qui n'appellent pas exit.

Le tableau récapitulatif de toutes les options implantées ainsi que les domaines de valeurs associés figurent en Annexe B.

Toute valeur hors domaine (ex. *--order=0*, *--order=101*, *--seed=-1*) provoque l'émission d'un message sur *stderr* et le retour du code *ERREUR_OPTION* (64), conformément aux exigences du sujet.

3.2 Tableau dynamique pour les successeurs

Deux implantations étaient envisageables pour stocker les successeurs d'un contexte. La liste chaînée offre une insertion en O(1), mais impose un parcours linéaire O(p) pour accéder à l'i-ème élément lors de la sélection aléatoire pénalisant d'autant la génération sur des contextes fréquents. Le tableau dynamique *ctrvalref* résout ce compromis : l'accès par indice est constant O(1), et l'insertion reste amortie constante grâce à la politique de doublement de capacité (*CAP_MUL = 2*).

4. Points forts

Le projet a été conçu dès l'origine selon les principes de la programmation modulaire stricte. Chaque module expose une interface publique via un fichier d'en-tête .h et masque entièrement son implantation dans le .c correspondant. Le cloisonnement public/privé est respecté sans exception : aucune structure interne n'est exposée à l'extérieur du module qui la définit. L'utilisateur de l'interface manipule des types opaques et passe systématiquement par les requêtes et commandes prévus à cet effet.

L'ensemble des références manipulées est de type générique *const void *.* Le qualificatif *const* garantit qu'aucune fonction ne peut modifier les données pointées par l'utilisateur, tandis que le type *void **** confère aux fonctions un caractère polymorphe : elles opèrent indifféremment sur n'importe quel type d'élément, sans dépendance vis-à-vis de la représentation concrète. Cette architecture confère à chaque module un rôle clair et exclusif, dont les bénéfices se complètent :

- **Module *randtext*** : centralise toute la logique algorithmique du projet. Le fichier *main.c* se limite à l'orchestration : toute modification de la stratégie d'implantation n'entraîne aucune retouche du point d'entrée.

- **Module *define*** : inclus par presque tous les autres, il rassemble macro-constantes et macro-fonctions du programme. Il supprime toute chaîne ou nombre magique, factorise le code et concentre la maintenance en un point unique : modifier une borne, un nom d'option ou un message revient à éditer une seule ligne.
- **Module *help*** : isole les fonctions d'affichage des messages informationnels. Sans lui, *option* et *randtext* seraient alourdis par des dizaines de lignes qui parasiteraient leur logique métier ; ce module externalise cette charge et préserve la lisibilité du reste du code.
- **Module *option*** : encapsule entièrement le traitement de la ligne de commande. La structure *option* n'est pas accessible directement depuis *main.c*, qui interroge les paramètres via les accesseurs *order*, *limit*, *size*, *mode* exposés par *randtext*. cela respecte strictement le principe d'encapsulation.

5. Analyse Théorique de Complexité

On note N le nombre de mots du texte source, M le nombre de mots à générer, n l'ordre de la génération (taille des clés) et L la longueur moyenne d'un mot.

Phase de construction

Pour chacun des N mots lus, trois opérations sont enchaînées : le hachage du mot (*str_hashfun*) en $O(L)$, le hachage et la comparaison du contexte composite via *context_hash* et *context_cmp* en $O(n \times L)$, et l'insertion du successeur dans le tableau dynamique en temps amorti constant $O(1)$. La complexité totale de la phase d'apprentissage est donc $O(N \times n \times L)$. Spatialement, chaque mot unique étant stocké une seule fois et la somme des tailles de tableaux de successeurs valant exactement N, l'empreinte mémoire est en $O(N \times L)$.

Phase de génération

Pour chacun des M mots produits, la recherche du contexte dans *h_word* coûte $O(n \times L)$, la sélection aléatoire du successeur est en $O(1)$ grâce à l'accès direct par indice dans le tableau dynamique, et le décalage du contexte est en $O(n)$. La complexité totale est $O(M \times n \times L)$. Si une liste chaînée avait été retenue, la sélection aléatoire aurait coûté $O(K_succ)$ par mot généré, où *K_succ* peut atteindre plusieurs milliers pour les mots grammaticaux fréquents ; le tableau dynamique supprime ce facteur.

Statistiques

Une campagne de mesures comparatives a été menée sur *lesmiserables.txt* aux ordres 1, 2, 3, 5 et 10 dans les trois modes d'affichage (*texte, table, words*), avec moyennes calculées sur 10 exécutions par configuration et profilage mémoire sous *Valgrind*. Le détail complet des mesures figure en Annexe C.

6. Limites

Surcoût de fragmentation des petites allocations

Lors de la phase d'apprentissage, la multiplication des appels à l'allocateur pour stocker les clés composites de contextes génère de nombreux petits blocs isolés en mémoire. Bien qu'ils soient tous correctement libérés, cela accroît légèrement l'empreinte mémoire totale.

Limites de main

Les mots sont obtenus par lecture sur l'entrée des suites consécutives de caractères ne relevant pas de la catégorie *isspace*, de longueur majorée par *SIZE_MAX* ; toute suite de tels caractères de longueur strictement supérieure à *SIZE_MAX* se retrouve donc découpée en plusieurs mots, ce qui peut altérer les statistiques de transition aux endroits concernés.

Limites du module *option*

- La cascade de blocs else { ... } imbriqués qui enchaîne les macros produit un code à indentation profondément déportée vers la droite. Ce style est volontaire (chaque macro émet un if qu'il faut faire suivre par un else pour qu'il ne se rattache pas au if suivant), mais il rend la lecture pénible. Une réorganisation en cascade plate (else if explicites, ou table de correspondance entre nom d'option et fonction de traitement) serait plus lisible si le temps le permet.
- Maintenir manuellement la longueur d'une chaîne fixe est fragile : si *--order* devient *--rang*, il faut penser à modifier *ORDER_LOPT_LEN*. Une alternative serait d'utiliser *strlen(XSTR(...))* à la compilation, mais cela complique les macros qui s'en servent.

Limites du module *define*

- Les identificateurs de macros destinés à la stringification (par exemple *ORDER_LOPT, ORDER_SOPT*) ne peuvent pas contenir d'espace, puisqu'ils sont pensés pour être convertis tels quels en littéraux de chaîne via *XSTR*, et non pour figurer en clair dans le code.

Limites du module *help*

- La lisibilité du code en pâtit, les blocs *printf* devenant longs et fragmentés. Le compromis a été assumé pour éviter la duplication de chaînes de caractères entre la logique d'analyse (*option.c*) et la documentation (*help.c*).

7. Conclusion

Le développement du programme *randtext* a constitué une mise en application directe et complète des concepts abordés en Algorithmique 3 : spécification de types de données abstraits, cloisonnement public/privé, structures de hachage, tableaux dynamiques et analyse de complexité. La conception modulaire stricte autour des modules *define*, *option, help* et *randtext* a permis de produire un code lisible, maintenable et conforme aux exigences éliminatoires du sujet.

Les choix d'implantation retenus ont été guidés par l'analyse théorique. L'unicité des chaînes en mémoire, l'usage d'un tableau dynamique à croissance géométrique pour les listes de successeurs, et la gestion uniforme du mot vide aux frontières ont permis d'obtenir une complexité maîtrisée linéaire en l'ordre, validé expérimentalement sur des corpus de plus de 650 000 mots.

Au-delà du livrable technique, ce projet a constitué une occasion précieuse de confronter une intuition algorithmique à des contraintes d'ingénierie, et d'apprendre à arbitrer entre simplicité d'implantation, performance et lisibilité du code.

8.Annexes

Annexe A : Liens entre les modules et schéma implantation

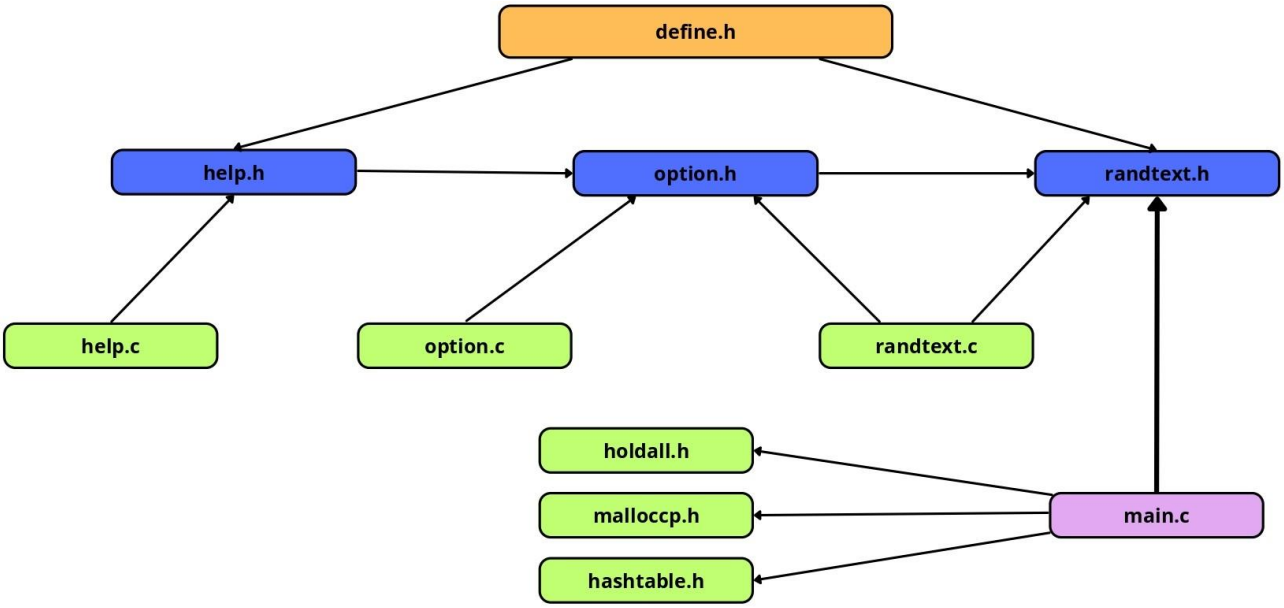


Figure 1 : schéma des liens entre les modules

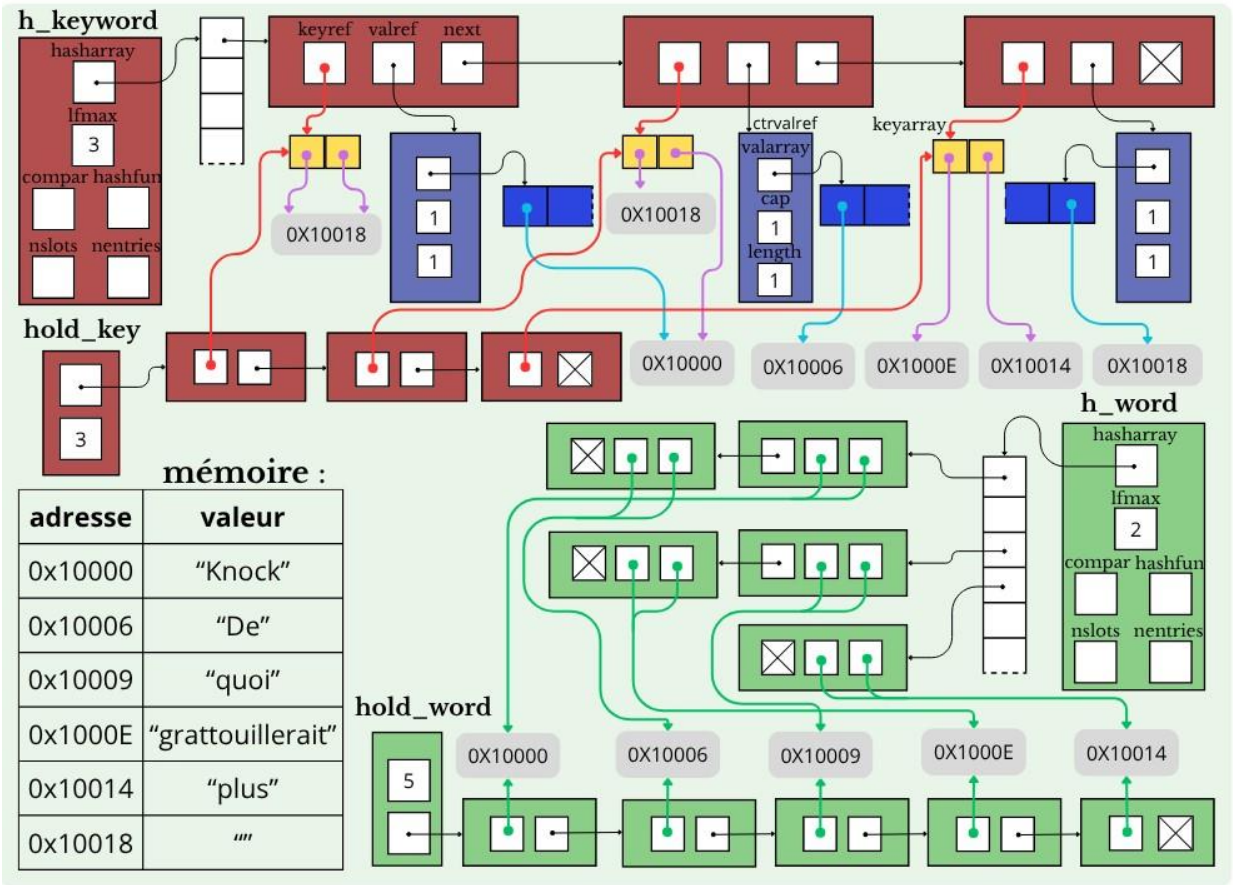


Figure 2 : schéma représentant l'implantation en mémoire

Annexe B : Tableau récapitulatif des options prises en charge

Option courte	Option longue	Domaine valide	Valeur par défaut	Valeurs testées et validées	Rôle
-o	--order	[1 – 100]	1	1, 2, 3, 5, 10, 50, 100	Ordre n de génération du texte.
-l	--limit	[0 – 4 294 967 295]	100	0, 1, 10, 50, 100, 500, 1 000, 10 000, 4 294 967 295	Nombre maximal de mots à produire.
-s	--seed	[0 – 4 294 967 295]	1	0, 1, 2, 42, 12 345, 99 999, 4 294 967 295	Germe du générateur pseudo-aléatoire.
-S	--size	[1 – 255]	63	1, 5, 31, 63, 255	Longueur maximale du préfixe significatif d'un mot.
-W	--width	[0 – 255]	0	0, 40, 80, 255	Largeur d'affichage en colonnes.
-r	--reveal=FEATURE	table / words	—	table, words	Affichage de la table clé-valeur ou la liste des mots distincts.
-t	--table	—	—	—	Alias de --reveal=table.
-w	--words	—	—	—	Alias de --reveal=words.
-?	--help	—	—	—	Affiche l'aide complète.
-M	--man	—	—	—	Affiche l'aide au format man.
-O	--options	—	—	—	Affiche les valeurs courantes des options.
-U	--usage	—	—	—	Affiche un résumé concis des options.
-V	--version	—	—	—	Affiche le nom, la version et la licence.

Annexe C : Tableau récapitulatif des statistiques

Mesures effectuées sur lesmiserables.txt — moyenne calculée sur 10 exécutions par configuration

Mode texte (sortie normale)

Ordre	Temps (ms)	Allocs	Octets
1	104	542 959	18,18 MB
2	346	1 677 627	35,70 MB
3	440	2 464 461	49,64 MB
5	540	2 726 103	61,73 MB
10	695	2 738 143	81,50 MB

Mode --reveal=table

Ordre	Temps (ms)	Allocs	Octets
1	144	542 958	18,18 MB
2	451	1 677 626	35,70 MB
3	623	2 464 460	49,64 MB
5	806	2 726 102	61,73 MB
10	1 076	2 738 142	81,50 MB

Mode --reveal=words

Ordre	Temps (ms)	Allocs	Octets
1	110	542 958	18,18 MB
2	351	1 677 626	35,70 MB
3	450	2 464 460	49,64 MB
5	550	2 726 102	61,73 MB
10	713	2 738 142	81,50 MB